Taller 1: Comparación de modelos de filtrado colaborativo en MovieLens 20M

Materia: Sistemas de Recomendación

Grupo: 05

Integrantes:

Alvaro Andres Castiblanco Lopez
Maestría en Ingeniería de la Información
a.castiblancol@uniandes.edu.co

Bryan Mauricio Guzmán García
Maestría en Arquitectura de Tecnologías de Información
b.guzmang@uniandes.edu.co

Juan Carlos Tovar Orjuela
Maestría en Ingeniería de la Información
jc.tovaro1@uniandes.edu.co

Lorraine Jazlady Rojas Parra
Maestría en Ingeniería de la Información
lj.rojasp1@uniandes.edu.co

Contents

1 Introducción **3**

2 Conocimiento del dataset de trabajo **3**

2.1 Estructura del dataset . . . 3

2.2 Vista general y calidad de datos . . . 4

2.3 Distribución global de ratings . . . 4

2.4 Análisis de usuarios . . . 5

2.5 Análisis de ítems . . . 6

2.6 Densidad, dispersión y cola larga . . . 7

2.7 Conclusión del análisis exploratorio . . . 7

3 Pre-procesamiento de datos **8**

3.1 Transformación de los datos a formatos compatibles con modelos colaborativos . . . 8

3.2 Muestreo de usuarios . . . 8

3.3 Partición en entrenamiento y prueba . . . 8

3.4 Validación de la partición . . . 9

3.5 Conclusión del pre-procesamiento . . . 9

4 Construcción de modelos colaborativos usuario–usuario **10**

4.1 Construcción de los modelos . . . 10

4.2 Predicción sobre el conjunto de prueba . . . 10

4.3 Evaluación global de las predicciones . . . 10

4.4 Distribución de las predicciones . . . 11

4.5 Conclusión parcial del enfoque usuario–usuario . . . 11

4.6 Impacto de la selección de vecinos . . . 11

4.6.1 Variación del número de vecinos . . . 11

4.6.2 Variación del umbral de similitud en Jaccard . . . 12

4.7 Ponderación por significancia de McLaughlin . . . 12

4.7.1 Jaccard con significance weighting . . . 13

4.7.2 Cosine y Pearson con significance weighting . . . 13

4.8 Conclusión del ajuste de parámetros . . . 14

5 Construcción de modelos colaborativos ítem–ítem **14**

5.1 Construcción de los modelos . . . 14

5.2 Predicciones sobre el conjunto de prueba . . . 14

5.3 Evaluación global . . . 14

5.4 Comparación con el enfoque usuario–usuario . . . 15

5.5 Impacto de la selección de vecinos . . . 15

5.5.1 Variación del número de vecinos . . . 15

5.5.2 Variación del umbral de similitud en Jaccard . . . 16

5.6 Ponderación por significancia de McLaughlin . . . 17

5.6.1 Jaccard con significance weighting . . . 17

5.6.2 Cosine y Pearson con significance weighting . . . 17

5.7 Conclusión del ajuste de parámetros en ítem–ítem . . . 17

6 Comparación final de los mejores modelos **18**

7 Conclusión final **18**

1. Introducción

Este informe presenta el desarrollo del **Taller 1** del curso de Sistemas de Recomendación, centrado en la construcción y evaluación de modelos de *filtrado colaborativo* usando el dataset *MovieLens 20M* [1]. El objetivo fue implementar modelos basados en similitud entre usuarios y entre ítems, y analizar su desempeño con distintas métricas y estrategias de selección de vecinos.

El trabajo se desarrolló en cuatro etapas. Primero, se realizó un análisis exploratorio del dataset para conocer su estructura, la calidad de los datos y las principales características de usuarios, ítems e interacciones. Luego, se llevó a cabo el preprocesamiento, incluyendo una estrategia de muestreo y una división temporal entre entrenamiento y prueba.

Después, se construyeron y evaluaron modelos **usuario–usuario** usando tres métricas de similitud: **Jaccard**, **coseno** y **Pearson**. En esta etapa se estudió tanto la calidad de las predicciones como la capacidad de identificar ítems relevantes, además del efecto de parámetros como el número de vecinos, el umbral de similitud y la ponderación por significancia.

Por último, se aplicó el mismo procedimiento a modelos **ítem–ítem** para comparar ambos enfoques y determinar cuál ofrece mejores resultados en este conjunto de datos. En general, el informe resume el comportamiento de distintos métodos clásicos de filtrado colaborativo en un escenario realista.

2. Conocimiento del dataset de trabajo

El taller se desarrolló sobre el dataset **MovieLens 20M**, el cual contiene interacciones explícitas entre usuarios y películas a través de ratings. Este conjunto de datos es apropiado para filtrado colaborativo porque representa directamente la relación usuario–ítem y permite construir una matriz de utilidad a partir de calificaciones reales.

2.1. Estructura del dataset

El dataset está compuesto por seis archivos, pero para el desarrollo del taller los más relevantes fueron `rating.csv` y `movie.csv`. El primero contiene la interacción entre usuarios e ítems, mientras que el segundo aporta la información descriptiva básica de cada película.

Tabla 1: Archivos del dataset MovieLens 20M y su función en el taller.

Archivo	Campos principales	Uso principal
`rating.csv`	`userId, movieId, rating, timestamp`	Interacciones usuario–ítem
`movie.csv`	`movieId, title, genres`	Metadatos de películas
`tag.csv`	`userId, movieId, tag, timestamp`	Etiquetas de usuario
`link.csv`	`movieId, imdbId, tmdbId`	Enlaces externos
`genome_scores.csv`	`movieId, tagId, relevance`	Relevancia película–tag
`genome_tags.csv`	`tagId, tag`	Catálogo de tags

En términos de modelado colaborativo, la interacción entre usuarios e ítems está representada en `rating.csv`, donde cada fila corresponde a un rating asignado por un usuario a una película. Por tanto, la estructura fundamental del problema está dada por la tupla:

$$(\texttt{userId}, \texttt{movieId}, \texttt{rating}, \texttt{timestamp})$$

A partir de esta tabla se puede construir la matriz de utilidad, donde las filas representan usuarios, las columnas representan películas y cada celda almacena el rating otorgado por un usuario a una película.

2.2. Vista general y calidad de datos

Se verificó que el dataset cargó correctamente, sin valores nulos en los campos principales y sin duplicados en la combinación usuario–película. Asimismo, todos los identificadores de películas presentes en `rating.csv` están contenidos en `movie.csv`, lo que garantiza consistencia entre interacciones y metadatos.

Tabla 2: Resumen de calidad de datos.

Aspecto	**Resultado**
Usuarios únicos	138,493
Películas en ratings	26,744
Películas en `movie.csv`	27,278
Rango de ratings	0.5 a 5.0
Valores nulos en ratings	0
Valores nulos en movies	0
Duplicados `userId-movieId`	0
`movieId` sin correspondencia en movies	0

Estos resultados muestran que el dataset tiene buena calidad estructural para el desarrollo del taller y no requiere correcciones mayores antes del preprocesamiento.

2.3. Distribución global de ratings

La distribución global de ratings no es uniforme. Los valores medios y altos concentran la mayor parte de las interacciones, especialmente los ratings de 3.0, 4.0 y 5.0. El rating más frecuente es 4.0, con aproximadamente el 27.81% de las observaciones, seguido por 3.0 con 21.46% y 5.0 con 14.49%.

Tabla 3: Distribución global de ratings.

Rating	**Frecuencia**	**Porcentaje (%)**
0.5	239,125	1.20
1.0	680,732	3.40
1.5	279,252	1.40
2.0	1,430,997	7.15
2.5	883,398	4.42
3.0	4,291,193	21.46
3.5	2,200,156	11.00
4.0	5,561,926	27.81
4.5	1,534,824	7.67
5.0	2,898,660	14.49

El rating promedio global es 3.53, con mediana 3.5 y desviación estándar cercana a 1.05. Esto indica una clara inclinación hacia valoraciones positivas o intermedias, lo cual es relevante para la posterior binarización de relevancia y para interpretar posibles sesgos en los modelos.

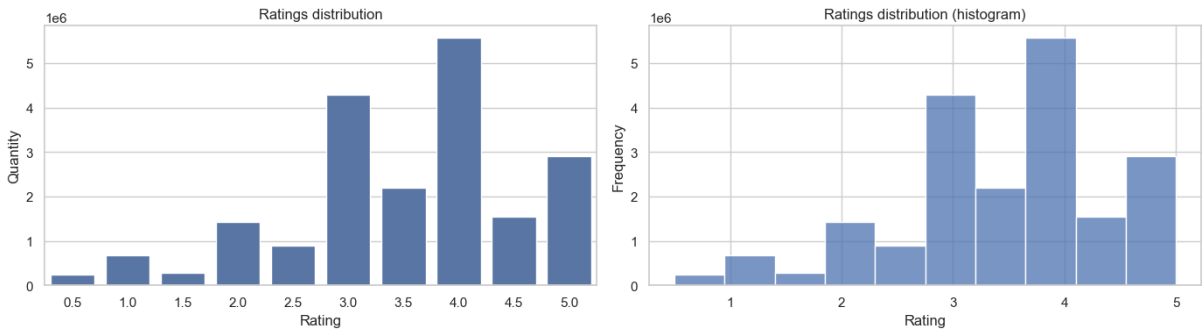

Figure 1: Distribución global de ratings en MovieLens 20M.

2.4. Análisis de usuarios

La actividad de los usuarios presenta una distribución fuertemente asimétrica. Aunque el mínimo de ratings por usuario es 20, la media es 144.41 y el máximo alcanza 9,254. La mediana es 68, lo que confirma la existencia de pocos usuarios extremadamente activos y una mayoría con actividad moderada.

Tabla 4: Resumen estadístico de actividad por usuario.

Métrica	**count**	**mean**	**std**	**min**	**50%**	**75%**	**max**
# ratings por usuario	138,493	144.41	230.27	20	68	155	9,254

El promedio de rating por usuario tiene media 3.63 y desviación estándar 0.44, lo que sugiere diferencias moderadas en la severidad de calificación de los usuarios. La varianza media por usuario es 0.96, lo que indica que, aunque existen diferencias individuales, en general los usuarios no usan toda la escala de ratings de forma extrema.

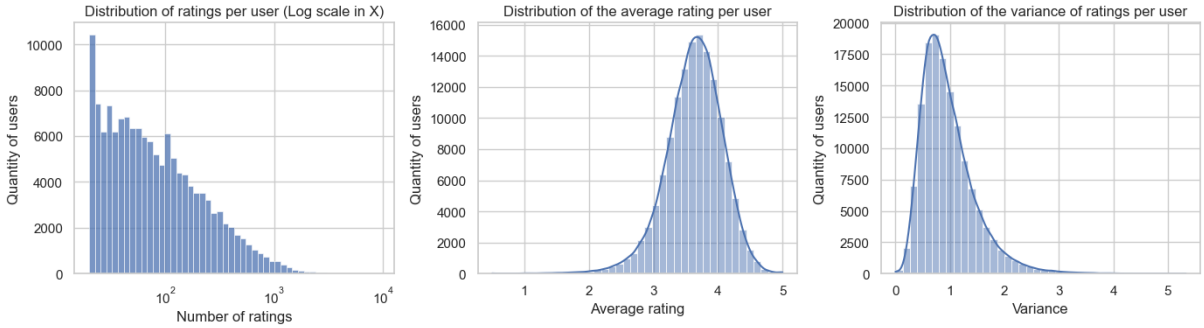

Figure 2: Análisis general de usuarios.

Tabla 5: Resumen estadístico del comportamiento de calificación por usuario.

Métrica	**count**	**mean**	**std**	**min**	**50%**	**75%**	**max**
Promedio de rating por usuario	138,493	3.63	0.44	0.50	3.65	3.92	5.00
Varianza de rating por usuario	138,493	0.96	0.49	0.00	0.87	1.20	5.33

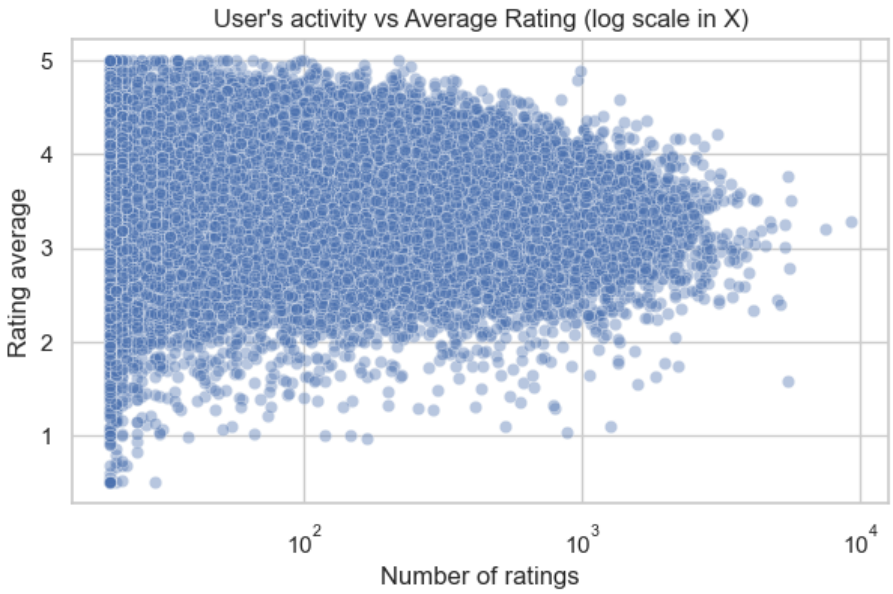

Figure 3: Actividad de usuarios vs promedio de rating.

2.5. Análisis de ítems

La popularidad de las películas también presenta una distribución de cola larga. Aunque existen 26,744 películas con ratings, la mediana de interacciones por ítem es solo 18, mientras que el máximo supera las 67 mil calificaciones. La media es 747.84, pero este valor está fuertemente influenciado por unas pocas películas extremadamente populares.

Tabla 6: Resumen estadístico de actividad por ítem.

Métrica	**count**	**mean**	**std**	**min**	**50%**	**75%**	**max**
# ratings por ítem	26,744	747.84	3085.82	1	18	205	67,310

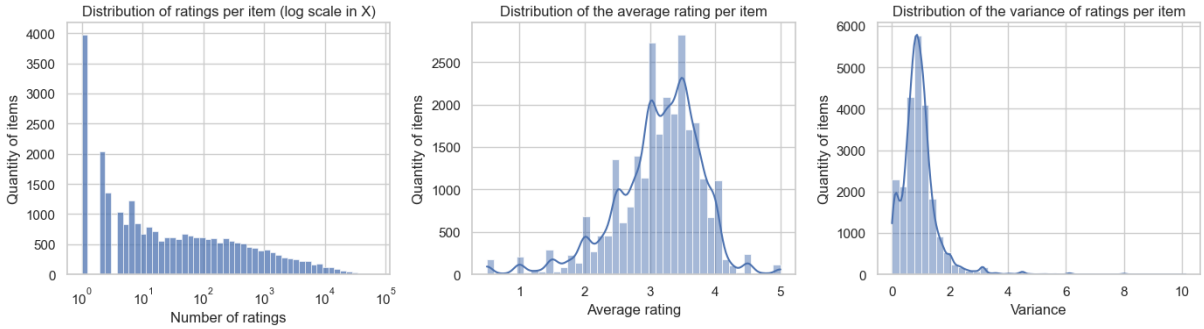

Figure 4: Análisis general de usuarios.

El promedio de rating por ítem es 3.13, con desviación estándar 0.66, mientras que la varianza media es 0.95. Esto indica que, aunque los ítems también presentan heterogeneidad, existe mayor concentración alrededor de calificaciones intermedias.

Tabla 7: Resumen estadístico del comportamiento de calificación por ítem.

Métrica	**count**	**mean**	**std**	**min**	**50%**	**75%**	**max**
Promedio de rating por ítem	26,744	3.13	0.66	0.50	3.24	3.57	5.00
Varianza de rating por ítem	22,772	0.95	0.65	0.00	0.88	1.14	10.13

Las películas con más ratings corresponden a títulos ampliamente conocidos y de alta popularidad, como *Pulp Fiction (1994)*, *Forrest Gump (1994)*, *The Shawshank Redemption (1994)* y *The Silence of the Lambs (1991)*.

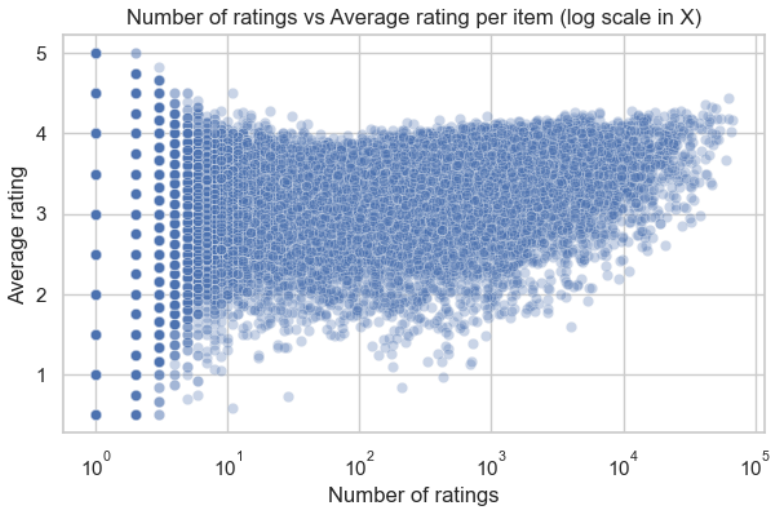

Figure 5: Número de ratings vs promedio de rating por ítem.

2.6. Densidad, dispersión y cola larga

A partir del número de usuarios, ítems e interacciones, se estimó la densidad de la matriz usuario–ítem:

$$\text{Densidad} = \frac{20{,}000{,}263}{138{,}493 \times 26{,}744} \approx 0.0054$$

lo que equivale a una dispersión de aproximadamente 99.46%. Esto confirma que se trata de una matriz altamente dispersa, una característica típica en sistemas de recomendación reales.

Tabla 8: Resumen global de la matriz usuario–ítem.

Medida	**Valor**
Usuarios	138,493
Ítems	26,744
Interacciones	20,000,263
Densidad	0.0054
Sparsity	0.9946

Además, la curva acumulada de popularidad muestra un fenómeno claro de *cola larga*: una pequeña fracción de películas concentra gran parte de los ratings. En particular:

- El top 10 de películas concentra el 2.91% de los ratings.
- El top 1% de ítems concentra el 33.48% de los ratings.
- El top 10% de ítems concentra el 86.05% de los ratings.

Esto implica que la mayoría de las interacciones se concentra en un subconjunto reducido de películas populares, mientras que una gran cantidad de ítems recibe muy pocas observaciones. Este fenómeno es crucial para el taller, ya que afecta directamente la construcción de vecinos, la cobertura de los modelos y la estabilidad de las similitudes colaborativas.

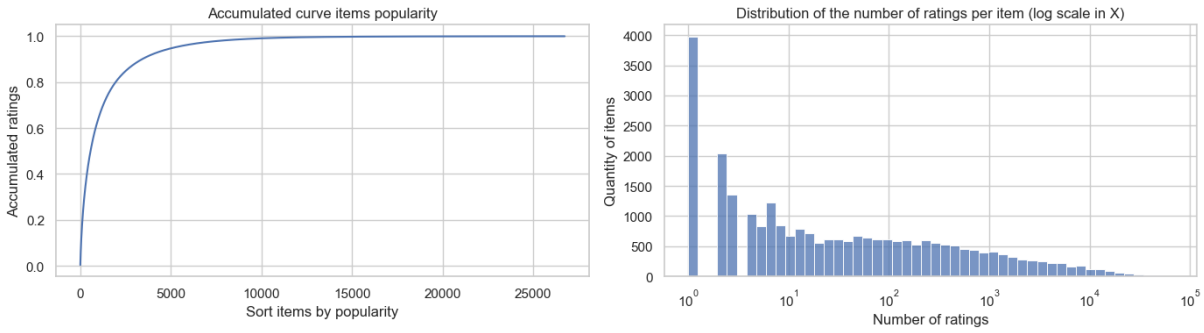

Figure 6: Evidencia del fenómeno de cola larga en la popularidad de ítems.

2.7. Conclusión del análisis exploratorio

El análisis del dataset permitió identificar varias propiedades importantes para el diseño del recomendador. Primero, se trata de un conjunto de datos limpio y consistente, adecuado para construir modelos colaborativos sin necesidad de una fase extensa de corrección. Segundo, la distribución de ratings está sesgada hacia valores medios y altos, lo que sugiere cautela al definir relevancia binaria. Tercero, tanto usuarios como ítems presentan distribuciones altamente asimétricas y un marcado fenómeno de cola larga. Finalmente, la matriz usuario–ítem es extremadamente dispersa, lo que anticipa desafíos en la construcción de similitudes y en la cobertura efectiva de los modelos de recomendación.

3. Pre-procesamiento de datos

Una vez comprendida la estructura del dataset, se realizó una fase de pre-procesamiento orientada a transformar las interacciones usuario–ítem en formatos compatibles con los modelos colaborativos vistos en clase. Esta etapa incluyó dos decisiones principales: (i) la construcción de representaciones explícitas y binarias a partir de los ratings, y (ii) la partición de los datos en conjuntos de entrenamiento y prueba.

3.1. Transformación de los datos a formatos compatibles con modelos colaborativos

El punto de partida fue el archivo de ratings, que contiene interacciones explícitas entre usuarios y películas en una escala de 0.5 a 5.0. A partir de esta información se construyeron tres versiones del conjunto de datos.

- **Ratings explícitos**: conserva la interacción original `(userId, movieId, rating, timestamp)` y se usa para modelos basados en similitud de ratings, como coseno y correlación de Pearson.
- **Ratings binarios**: transforma cada rating en una etiqueta de relevancia `relevant`, definida como 1 si el rating es mayor o igual a 3.5 y 0 en caso contrario. Esta versión permite evaluar el problema como una tarea de clasificación de relevancia.
- **Ratings binarios positivos**: conserva únicamente los pares `(userId, movieId)` cuya relevancia es positiva. Esta representación es especialmente útil para modelos basados en conjuntos, como el índice de Jaccard.

La elección del umbral de 3.5 se justificó por dos razones. Primero, la distribución global de ratings mostró una fuerte concentración en valores intermedios y altos, por lo que este umbral permite separar de manera razonable las interacciones claramente favorables de las no favorables. Segundo, este criterio es consistente con la interpretación usual de relevancia en sistemas de recomendación explícitos, donde ratings altos se consideran evidencia de preferencia.

Tabla 9: Versiones derivadas del dataset para el modelado colaborativo.

Versión	Estructura	Uso principal
`ratings_explicit`	`userId, movieId, rating, timestamp`	Cosine, Pearson
`ratings_binary`	`userId, movieId, rating, relevant`	Evaluación binaria
`ratings_binary_positive`	`userId, movieId`	Jaccard

Esta transformación permitió trabajar con una misma base de interacciones bajo diferentes perspectivas: predicción numérica de rating y predicción binaria de relevancia, según las necesidades de cada modelo.

3.2. Muestreo de usuarios

Dado el tamaño del dataset original y las limitaciones de memoria del entorno de cómputo, se decidió reducir el volumen de datos antes de entrenar los modelos. En lugar de muestrear interacciones individuales, se optó por muestrear **usuarios completos**, preservando todas sus interacciones. Esta decisión fue importante porque los modelos colaborativos construidos en el taller dependen de la coherencia del perfil de cada usuario y de su historial de consumo.

El muestreo se realizó de forma **estratificada por nivel de actividad**. Para ello, los usuarios se agruparon según su número de ratings y luego se construyeron muestras del 10%, 30% y 40% de usuarios, manteniendo la proporción de usuarios poco activos, medianamente activos y altamente activos en cada caso. Posteriormente, se compararon estas muestras frente al dataset completo en términos de actividad y comportamiento promedio de rating.

A partir de este análisis se seleccionó como muestra de trabajo la del **30% de usuarios**, ya que ofrecía un equilibrio adecuado entre representatividad estadística y viabilidad computacional. Esta muestra fue la base para todos los experimentos posteriores del taller.

3.3. Partición en entrenamiento y prueba

Sobre la muestra seleccionada se realizó una partición **temporal por usuario**. En esta estrategia, las interacciones de cada usuario se ordenaron cronológicamente por `timestamp`, asignando aproximadamente el 80% más antiguo al conjunto de entrenamiento y el 20% más reciente al conjunto de prueba.

Esta decisión fue preferible a un split aleatorio global por varias razones. En primer lugar, garantiza que cada usuario evaluado en test conserva parte de su historial en train, lo cual es indispensable para modelos colaborativos basados en perfiles. En segundo lugar, evita fuga de información, ya que las predicciones se realizan usando únicamente interacciones pasadas del usuario. Finalmente, representa de forma más realista el problema de recomendación, donde el sistema aprende del comportamiento histórico y predice preferencias futuras.

Tabla 10: Resumen de la estrategia de partición de datos.

Aspecto	**Decisión**
Unidad de muestreo	Usuarios completos
Tipo de muestreo	Estratificado por actividad
Muestra de trabajo	30% de usuarios
Tipo de split	Temporal por usuario
Proporción train/test	80% / 20%
Criterio temporal	Interacciones más antiguas a train, más recientes a test

A partir de esta partición se derivaron los conjuntos:

- `ratings_explicit_train` y `ratings_explicit_test`
- `ratings_binary_train` y `ratings_binary_test`
- `ratings_binary_positive_train` y `ratings_binary_positive_test`

Todos estos archivos se almacenaron por separado en formato `Parquet`, lo que permitió reutilizarlos eficientemente en las siguientes fases del taller.

3.4. Validación de la partición

La partición obtenida fue consistente con los objetivos del modelado. Todos los usuarios presentes en el conjunto de prueba también aparecieron en entrenamiento, no hubo traslape de pares usuario–ítem entre train y test, y no se detectaron violaciones del orden temporal. Además, cada usuario conservó suficientes interacciones en ambos conjuntos para permitir tanto la construcción del perfil como la evaluación posterior.

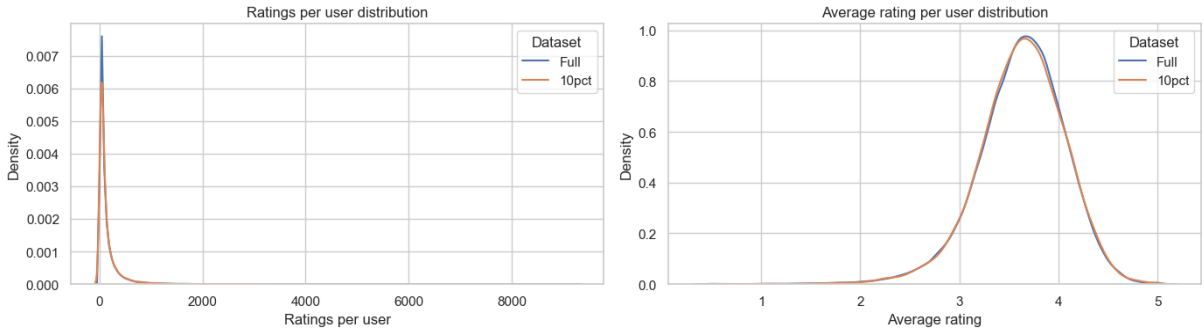

Figure 7: Validación de la partición.

Tabla 11: Resumen de validación del split temporal.

Validación	**Resultado**
Usuarios en train	138,493
Usuarios en test	138,493
Usuarios en test no presentes en train	0
Usuarios con split temporal inválido	0
Usuarios con menos de 5 interacciones en train	0
Usuarios con menos de 1 interacción en test	0

3.5. Conclusión del pre-procesamiento

El pre-procesamiento permitió convertir el dataset original en representaciones compatibles con distintos modelos colaborativos, manteniendo consistencia entre la perspectiva explícita y la binaria. Además, la estrategia de muestreo y partición temporal permitió reducir el volumen de datos a un tamaño manejable sin romper la lógica del problema de recomendación. Como resultado, se obtuvo una base de entrenamiento y evaluación adecuada para construir, comparar y ajustar modelos usuario–usuario e ítem–ítem en las siguientes secciones del taller.

4. Construcción de modelos colaborativos usuario–usuario

En esta sección se construyeron y evaluaron modelos colaborativos basados en perfiles de usuario utilizando el conjunto de entrenamiento definido en la fase de pre-procesamiento. El objetivo fue comparar tres estrategias de similitud entre usuarios: **Índice de Jaccard**, **distancia coseno** y **correlación de Pearson**. A partir de estos modelos se generaron predicciones sobre el conjunto de prueba y se evaluó su calidad tanto en términos de predicción de rating como de predicción de relevancia.

4.1. Construcción de los modelos

Se construyeron tres modelos colaborativos usuario–usuario, todos basados en la misma lógica general: para predecir la preferencia de un usuario sobre un ítem, se identifican vecinos similares y se agrega su información para estimar la relevancia o el rating esperado.

- **Jaccard usuario–usuario**: se implementó una versión manual basada en una representación binaria del historial de cada usuario. Cada perfil se definió como el conjunto de ítems relevantes consumidos en entrenamiento, y la similitud entre dos usuarios se calculó como la razón entre la intersección y la unión de dichos conjuntos.
- **Cosine usuario–usuario**: se construyó sobre ratings explícitos utilizando la librería `Surprise` [2], configurando el algoritmo KNN en modo `user_based=True`.
- **Pearson usuario–usuario**: también se implementó con `Surprise`, utilizando correlación de Pearson entre perfiles de usuario y una variante con medias para estabilizar la predicción.

En los tres casos, el modelo se entrenó exclusivamente con el conjunto `train`, mientras que el conjunto `test` se reservó para la evaluación posterior.

4.2. Predicción sobre el conjunto de prueba

A partir de los modelos entrenados se generaron predicciones sobre la segunda parte de los datos. En el caso de Jaccard, la predicción de relevancia se realizó directamente sobre el espacio binario, identificando usuarios vecinos que hubieran interactuado con el ítem objetivo y agregando sus etiquetas de relevancia ponderadas por similitud.

En los casos de Cosine y Pearson, el procedimiento se realizó con la funcionalidad nativa de `Surprise`, que predice ratings explícitos para cada par usuario–ítem del conjunto de prueba. Posteriormente, estos ratings predichos se transformaron a relevancia binaria utilizando el mismo umbral empleado en el pre-procesamiento, es decir, una predicción se consideró relevante si el rating estimado era mayor o igual a 3.5.

Los tres modelos generaron predicciones sobre exactamente 407,538 interacciones del conjunto de prueba, lo que permitió una comparación directa bajo las mismas condiciones experimentales.

4.3. Evaluación global de las predicciones

La evaluación se realizó en dos niveles. En primer lugar, para los modelos basados en Cosine y Pearson, que producen ratings estimados, se calcularon las métricas **RMSE** y **MAE**. Estas métricas permiten medir qué tan lejos se encuentra el rating predicho respecto al rating real observado en el dataset. En segundo lugar, para los tres modelos se evaluó la capacidad de predicción de relevancia binaria mediante **accuracy**, **precision**, **recall** y **F1-score**.

Tabla 12: Resultados globales de los modelos usuario–usuario.

Modelo	Accuracy	Precision	Recall	F1	RMSE	MAE
Jaccard	0.5992	0.5988	0.9976	0.7484	–	–
Cosine	0.6864	0.7218	0.7730	0.7465	0.9494	0.7265
Pearson	0.7037	0.7794	0.7029	0.7392	0.8804	0.6700

Los resultados muestran que los tres modelos exhiben comportamientos distintos. El modelo basado en Jaccard alcanzó un **recall extremadamente alto** (0.9976), lo cual indica que casi todos los ítems realmente relevantes fueron clasificados como positivos. Sin embargo, este comportamiento estuvo acompañado de una fuerte sobrepredicción de la clase positiva, lo que redujo la precisión y limitó su capacidad discriminativa.

Por su parte, el modelo basado en Cosine presentó el mejor balance entre *precision* y *recall*, alcanzando el mayor equilibrio global en términos de clasificación de relevancia. Su F1-score fue ligeramente inferior al de Jaccard, pero con una distribución de predicciones mucho más razonable y una mejor separación entre casos positivos y negativos.

Finalmente, el modelo basado en Pearson obtuvo los mejores resultados en predicción de rating, con el menor RMSE y el menor MAE. Esto indica que fue el modelo más preciso al aproximar la calificación real del usuario. Además, logró la mayor accuracy y la mayor precision entre los tres modelos, aunque con un recall menor que Cosine y Jaccard.

4.4. Distribución de las predicciones

La distribución de relevancia predicha también permitió entender mejor el comportamiento de cada modelo. En el caso de Jaccard, el 99.54% de las predicciones fueron clasificadas como relevantes, lo que explica su recall casi perfecto y su baja capacidad de discriminación. En contraste, Cosine y Pearson mostraron distribuciones más balanceadas:

- **Jaccard**: 99.54% de predicciones positivas
- **Cosine**: 63.98% de predicciones positivas
- **Pearson**: 53.88% de predicciones positivas

Esto confirma que Jaccard opera como un modelo mucho más permisivo, mientras que Pearson resulta más conservador y Cosine se sitúa en un punto intermedio.

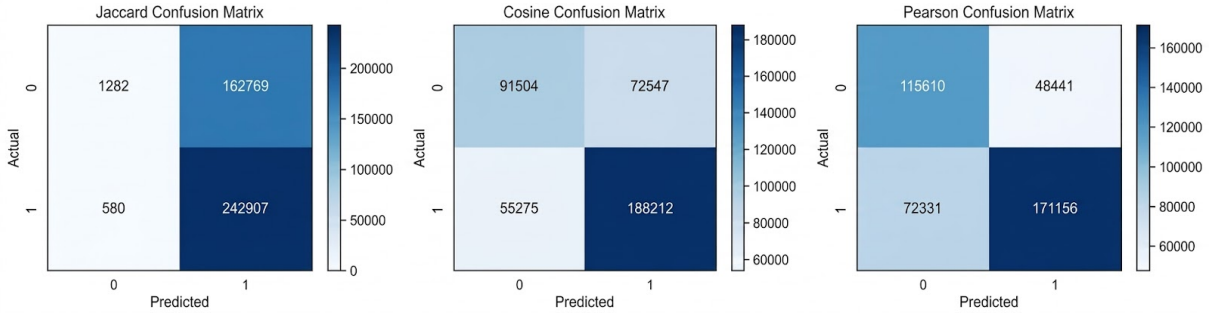

Figure 8: Matrices de confusión de los modelos usuario–usuario.

4.5. Conclusión parcial del enfoque usuario–usuario

En términos generales, los resultados muestran que el enfoque usuario–usuario funciona razonablemente bien sobre el dataset. No obstante, la elección de la medida de similitud influye de manera importante en el comportamiento del sistema. Jaccard destaca por su capacidad de recuperación, pero sufre una marcada sobrepredicción positiva. Cosine ofrece el mejor compromiso global para clasificación de relevancia, mientras que Pearson es superior cuando el objetivo es aproximar con mayor precisión el rating real del usuario.

En consecuencia, si el problema se interpreta como una tarea de **predicción de rating**, el modelo más sólido en esta etapa es **Pearson usuario–usuario**. Si, por el contrario, se prioriza la **predicción balanceada de relevancia**, el modelo basado en **Cosine usuario–usuario** resulta una alternativa especialmente competitiva.

4.6. Impacto de la selección de vecinos

Una vez evaluados los modelos base, se analizó el efecto de variar la estrategia de selección de vecinos. En particular, se estudió el impacto del número máximo de vecinos k y, en el caso del modelo Jaccard, del umbral mínimo de similitud requerido para aceptar un vecino.

4.6.1. Variación del número de vecinos

Para los modelos basados en Cosine y Pearson se evaluaron distintos valores de k, observando su efecto sobre RMSE, MAE y F1-score. En ambos casos, aumentar el número de vecinos produjo mejoras consistentes en las métricas de rating y en la calidad de la clasificación de relevancia, aunque con rendimientos decrecientes a partir de valores medios de k.

Tabla 13: Impacto del número de vecinos en los modelos usuario–usuario basados en Cosine y Pearson.

Modelo	k	RMSE	MAE	F1	Vecinos prom.	% sin vecinos
Cosine	5	1.0382	0.7992	0.7094	4.95	0.0000
Cosine	10	0.9894	0.7590	0.7289	9.79	0.0000
Cosine	20	0.9624	0.7375	0.7419	19.23	0.0000
Cosine	40	0.9471	0.7247	0.7487	37.32	0.0000
Cosine	80	0.9388	0.7178	0.7496	71.07	0.0000
Pearson	5	0.9602	0.7376	0.7144	4.93	0.0016
Pearson	10	0.9181	0.7038	0.7267	9.74	0.0016
Pearson	20	0.8943	0.6835	0.7346	19.07	0.0016
Pearson	40	0.8824	0.6733	0.7393	36.82	0.0016
Pearson	80	0.8772	0.6681	0.7422	69.46	0.0016

Los resultados muestran que ambos modelos se benefician de incorporar más vecinos, especialmente al pasar de valores bajos de k a valores intermedios. Sin embargo, a partir de $k = 40$ la mejora marginal es reducida, lo que sugiere la existencia de un punto de equilibrio entre calidad predictiva y costo computacional.

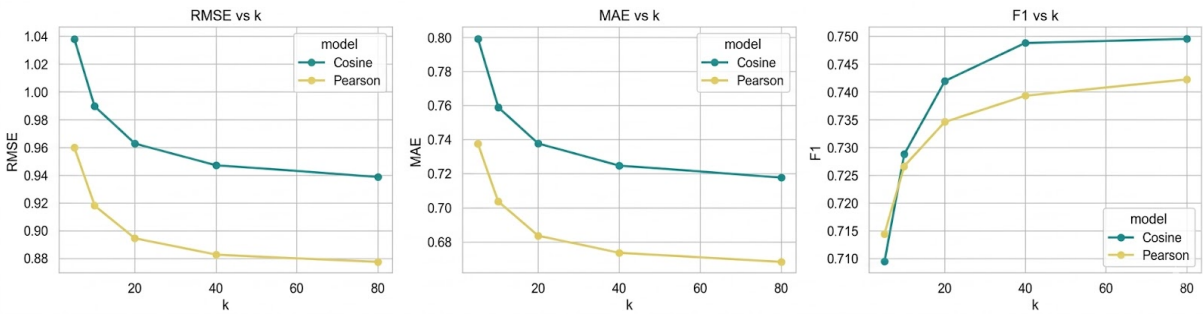

Figure 9: Impacto del número de vecinos k sobre RMSE, MAE y F1-score en los modelos usuario–usuario basados en Cosine y Pearson.

4.6.2. Variación del umbral de similitud en Jaccard

En el caso del modelo Jaccard, variar el número de vecinos no produjo cambios relevantes en las métricas, por lo que el análisis se concentró en el umbral mínimo de similitud. Este parámetro controla qué tan estricta debe ser la coincidencia entre usuarios para que uno sea considerado vecino del otro.

Tabla 14: Impacto del umbral de similitud en el modelo Jaccard usuario–usuario.

Umbral	Precision	Recall	F1	Vecinos prom.	% sin vecinos
0.0	0.5990	0.9977	0.7486	34.80	0.0138
0.1	0.6086	0.9789	0.7506	24.55	0.1089
0.2	0.6492	0.9013	0.7548	5.88	0.5271
0.3	0.6693	0.8349	0.7430	0.49	0.9244

A medida que aumenta el umbral, el modelo se vuelve más selectivo: mejora la precision, disminuye el recall y aumenta la proporción de casos sin vecinos. El mejor F1-score se obtuvo alrededor de `min_similarity` = 0.2, donde se logró un mejor equilibrio entre recuperación y capacidad discriminativa.

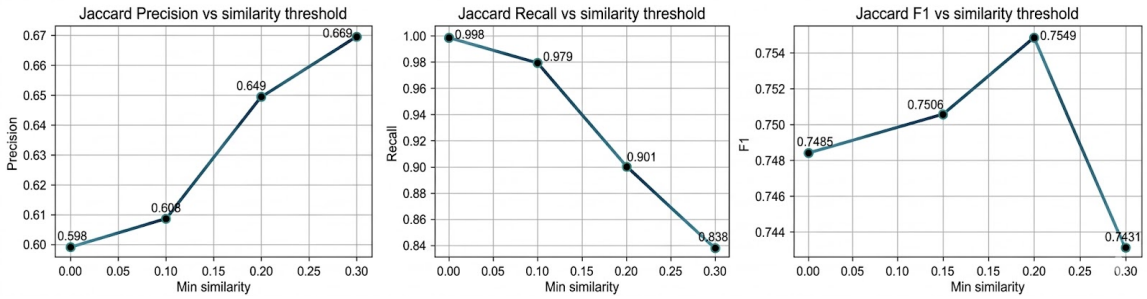

Figure 10: Impacto del umbral de similitud sobre precision, recall y F1-score en el modelo Jaccard usuario–usuario.

4.7. Ponderación por significancia de McLaughlin

Posteriormente, se estudió la estrategia de *significance weighting* propuesta por McLaughlin [3], cuyo objetivo es penalizar similitudes calculadas sobre poca evidencia compartida. La intuición es que una similitud alta entre dos usuarios es más confiable si ambos han coincidido en un número amplio de ítems, y menos confiable si dicha coincidencia se basa en unos pocos elementos.

La estrategia utilizada consistió en ajustar la similitud original multiplicándola por un factor de significancia de la forma:

$$w_{uv} = \min\left(1, \frac{n_{uv}}{\beta}\right), \qquad sim'_{uv} = sim_{uv} \cdot w_{uv},$$

donde n_{uv} representa el número de ítems compartidos entre usuarios y β controla la intensidad de la penalización.

4.7.1. Jaccard con significance weighting

En el modelo Jaccard, la ponderación por significancia modificó principalmente el balance entre precision y recall, aunque el F1-score permaneció prácticamente estable.

Tabla 15: Impacto de la significance weighting en Jaccard usuario–usuario.

β	**Precision**	**Recall**	**F1**	**Vecinos prom.**	**% sin vecinos**
5	0.6437	0.8984	0.7500	5.84	0.5218
10	0.6456	0.8954	0.7502	5.66	0.5305
20	0.6496	0.8873	0.7501	5.10	0.5579
40	0.6558	0.8764	0.7502	4.44	0.6118

Los resultados muestran que, al aumentar β, el modelo se vuelve más conservador: incrementa la precision, reduce el recall y disminuye el número de vecinos utilizados. Sin embargo, el F1-score se mantiene casi constante, lo que indica que la significance weighting en Jaccard sirve más como mecanismo de regulación del comportamiento que como una mejora sustancial del desempeño global.

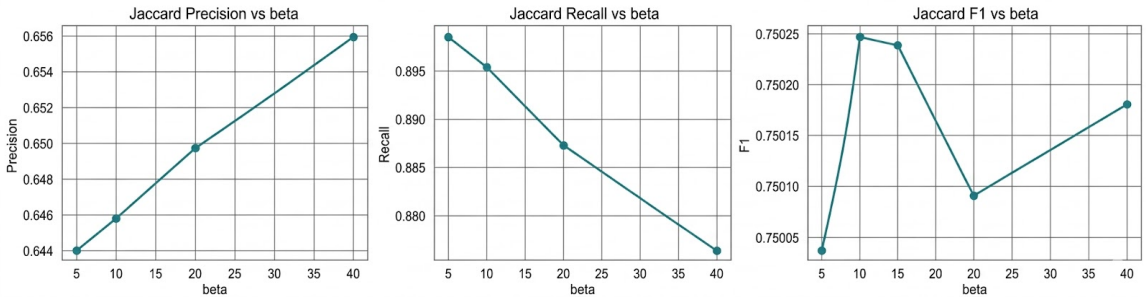

Figure 11: Impacto de β sobre precision, recall y F1-score en Jaccard usuario–usuario con significance weighting.

4.7.2. Cosine y Pearson con significance weighting

En los modelos basados en Cosine y Pearson, la significance weighting tuvo un efecto diferenciado. En Cosine se observó una mejora leve con valores moderados de β, especialmente en F1-score, mientras que en Pearson la estrategia produjo una mejora más clara y consistente tanto en predicción de rating como en relevancia.

Tabla 16: Impacto de la significance weighting en los modelos Cosine y Pearson usuario–usuario.

Modelo	β	**RMSE**	**MAE**	**F1**	**Vecinos prom.**	**% sin vecinos**
Cosine	5	0.9536	0.7270	0.7566	37.13	0.0071
Cosine	10	0.9500	0.7242	0.7576	37.13	0.0071
Cosine	20	0.9489	0.7234	0.7547	37.13	0.0071
Cosine	40	0.9503	0.7264	0.7463	37.13	0.0071
Pearson	5	0.8771	0.6705	0.7425	36.64	0.0090
Pearson	10	0.8727	0.6666	0.7451	36.64	0.0090
Pearson	20	0.8694	0.6621	0.7527	36.64	0.0090
Pearson	40	0.8701	0.6618	0.7574	36.64	0.0090

En Cosine, los mejores resultados se obtuvieron para valores intermedios de β, lo que sugiere que una penalización moderada puede eliminar vecinos poco confiables sin perder demasiada cobertura. En Pearson, la mejora fue más marcada: tanto RMSE como MAE disminuyeron y el F1-score aumentó respecto al baseline, lo que indica que la estrategia fue particularmente útil para este modelo.

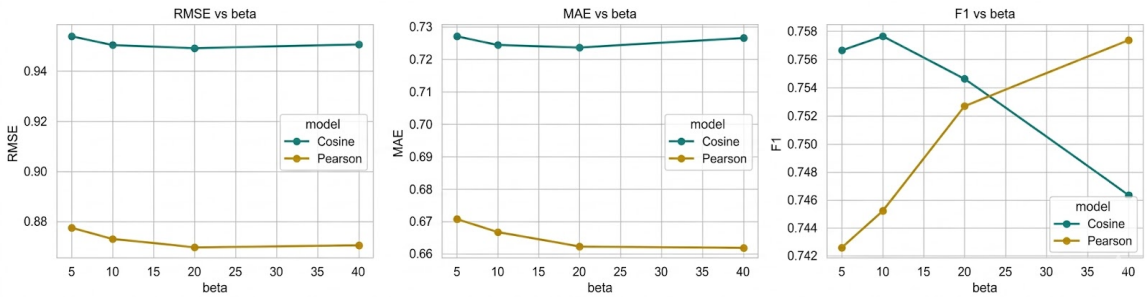

Figure 12: Impacto de β sobre RMSE, MAE y F1-score en los modelos Cosine y Pearson usuario–usuario con significance weighting.

4.8. Conclusión del ajuste de parámetros

Los experimentos de sensibilidad muestran que el impacto de los parámetros depende del modelo considerado. En Jaccard, el parámetro más relevante fue el umbral de similitud, mientras que el número de vecinos tuvo un efecto prácticamente nulo. En Cosine y Pearson, por el contrario, aumentar el número de vecinos produjo mejoras claras hasta un punto de estabilización alrededor de $k = 40$.

Por su parte, la estrategia de significance weighting no afectó de la misma forma a todos los modelos. En Jaccard su efecto fue limitado y se concentró en modificar el balance entre precision y recall. En Cosine produjo mejoras leves bajo penalización moderada. En Pearson, en cambio, la ponderación por significancia resultó claramente beneficiosa y permitió obtener la mejor combinación entre precisión de rating y calidad de relevancia dentro del enfoque usuario–usuario.

En conjunto, estos resultados muestran que el modelo **Pearson usuario–usuario con significance weighting** constituye la configuración más sólida de esta familia, mientras que **Cosine usuario–usuario** sigue siendo una alternativa muy competitiva cuando se prioriza equilibrio y simplicidad.

5. Construcción de modelos colaborativos ítem–ítem

En esta sección se replicó la metodología del punto anterior, pero construyendo vecindarios entre ítems en lugar de usuarios. Dado que el procedimiento general de partición, predicción y evaluación ya fue establecido para el enfoque usuario–usuario, aquí se presentan de forma más compacta únicamente los cambios de implementación y los resultados más relevantes.

5.1. Construcción de los modelos

Se construyeron tres modelos colaborativos ítem–ítem:

- **Jaccard ítem–ítem**: se implementó una variante manual en la que cada película se representó como el conjunto de usuarios que la marcaron como relevante en entrenamiento. La similitud entre dos ítems se calculó mediante intersección sobre unión de esos conjuntos.
- **Cosine ítem–ítem**: se construyó con `Surprise` usando un modelo KNN con `user_based=False`.
- **Pearson ítem–ítem**: se construyó también con `Surprise`, usando correlación de Pearson entre ítems y configuración `user_based=False`.

En todos los casos, la predicción para un par usuario–ítem se obtuvo a partir de ítems vecinos que el usuario había consumido previamente en el conjunto de entrenamiento.

5.2. Predicciones sobre el conjunto de prueba

A partir de los modelos entrenados se generaron predicciones sobre el conjunto de test. En Jaccard, la predicción de relevancia se obtuvo directamente sobre el espacio binario, agregando la señal de ítems vecinos relevantes del historial del usuario. En Cosine y Pearson, se predijeron ratings explícitos y luego se transformaron a relevancia binaria con el mismo umbral definido previamente ($rating \geq 3.5$).

5.3. Evaluación global

La evaluación se realizó con el mismo esquema usado en el enfoque usuario–usuario. Para Cosine y Pearson se calcularon **RMSE** y **MAE**, mientras que para los tres modelos se evaluó la capacidad de predicción de relevancia mediante **accuracy**, **precision**, **recall** y **F1-score**.

Tabla 17: Resultados globales de los modelos ítem–ítem.

Modelo	Accuracy	Precision	Recall	F1	RMSE	MAE
Jaccard	0.5992	0.5988	0.9976	0.7484	–	–
Cosine	0.6708	0.7004	0.7862	0.7408	0.9593	0.7279
Pearson	0.7222	0.7925	0.7259	0.7578	0.8444	0.6393

Los resultados muestran nuevamente comportamientos diferenciados entre los modelos. El modelo Jaccard mantiene el mismo patrón observado en usuario–usuario: un **recall extremadamente alto** acompañado de una fuerte sobrepredicción de la clase positiva, lo que limita su capacidad discriminativa. En este caso, el 99.54% de las predicciones fueron clasificadas como relevantes, lo que confirma que el modelo sigue siendo demasiado permisivo.

El modelo Cosine presentó un comportamiento intermedio, con una distribución de predicciones más balanceada que Jaccard y un desempeño razonable tanto en rating como en relevancia. Sin embargo, sus métricas quedaron por debajo de Pearson en casi todos los indicadores principales.

El mejor desempeño dentro del enfoque ítem–ítem se obtuvo con **Pearson**, que alcanzó el menor RMSE y el menor MAE, además del mejor accuracy, precision y F1-score. Esto indica que la similitud entre películas basada en correlación de Pearson logró capturar de forma más precisa la estructura de preferencias del dataset.

Tabla 18: Distribución de relevancia predicha por los modelos ítem–ítem.

Modelo	% predicciones no relevantes	% predicciones relevantes
Jaccard	0.46	99.54
Cosine	32.82	67.18
Pearson	45.18	54.82

La tabla anterior muestra que Pearson fue también el modelo más balanceado en términos de clasificación, mientras que Jaccard continuó sesgado casi por completo hacia la clase positiva.

5.4. Comparación con el enfoque usuario–usuario

Al comparar directamente ambos enfoques, se observa que:

- **Jaccard** exhibió prácticamente el mismo comportamiento en usuario–usuario e ítem–ítem, lo que sugiere que su principal limitación no depende de la orientación del vecindario, sino de la propia naturaleza binaria y de la estrategia de agregación.
- **Cosine usuario–usuario** fue ligeramente superior a **Cosine ítem–ítem**, especialmente en accuracy, precision y F1.
- **Pearson ítem–ítem** superó claramente a **Pearson usuario–usuario** en todas las métricas principales, tanto de rating como de relevancia.

En consecuencia, dentro de los modelos evaluados hasta este punto, el mejor comportamiento global se obtuvo con **Pearson ítem–ítem**, lo que sugiere que, para este dataset, la estructura de similitud entre ítems resulta más estable y más informativa que la similitud entre usuarios.

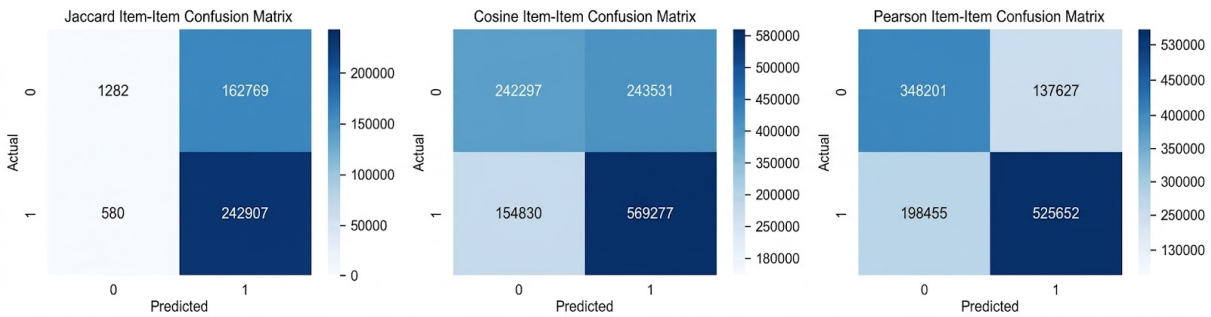

Figure 13: Matrices de confusión de los modelos ítem–ítem.

5.5. Impacto de la selección de vecinos

Al igual que en el enfoque usuario–usuario, se analizó el efecto del número de vecinos k y, en el caso del modelo Jaccard, del umbral mínimo de similitud. Dado que la metodología ya fue presentada en la sección anterior, en este caso se utilizó una rejilla de parámetros más compacta.

5.5.1. Variación del número de vecinos

Para los modelos Cosine y Pearson ítem–ítem se evaluaron valores de k iguales a 10, 40 y 80. Los resultados muestran que ambos modelos mejoran al aumentar el número de vecinos desde valores bajos hasta un rango intermedio, pero a partir de $k = 40$ los beneficios adicionales son pequeños.

Tabla 19: Impacto del número de vecinos en los modelos ítem–ítem basados en Cosine y Pearson.

Modelo	k	RMSE	MAE	F1	Vecinos prom.	% sin vecinos
Cosine	10	0.9863	0.7485	0.7254	10.00	0.0000
Cosine	40	0.9568	0.7264	0.7424	38.81	0.0000
Cosine	80	0.9561	0.7272	0.7387	72.82	0.0000
Pearson	10	0.8716	0.6606	0.7510	9.96	0.0032
Pearson	40	0.8403	0.6372	0.7582	38.41	0.0032
Pearson	80	0.8379	0.6355	0.7580	71.58	0.0032

En ambos modelos, el salto de $k = 10$ a $k = 40$ produjo mejoras visibles en RMSE, MAE y F1-score. Sin embargo, al pasar de $k = 40$ a $k = 80$, la mejora marginal fue muy reducida. Esto indica que $k = 40$ constituye un buen punto de equilibrio entre calidad predictiva y costo computacional.

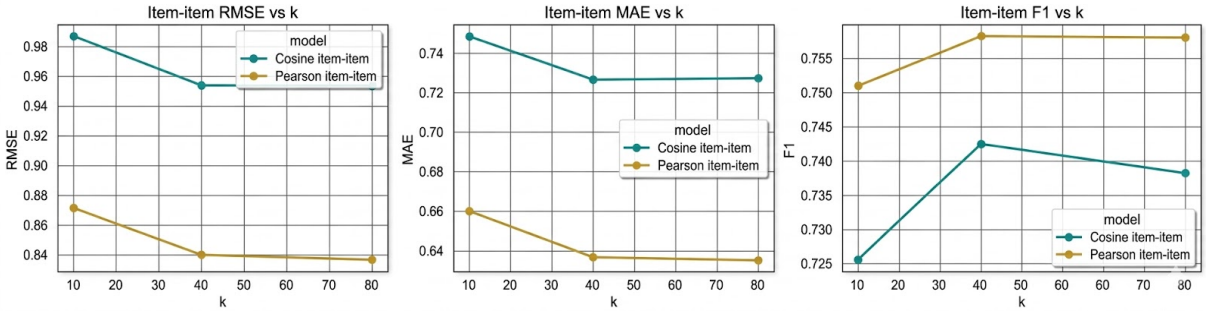

Figure 14: Impacto del número de vecinos k sobre RMSE, MAE y F1-score en los modelos ítem–ítem basados en Cosine y Pearson.

5.5.2. Variación del umbral de similitud en Jaccard

En el caso de Jaccard ítem–ítem, variar el número de vecinos no produjo cambios relevantes en las métricas. Por ello, el análisis se concentró en el umbral mínimo de similitud.

Tabla 20: Impacto del umbral de similitud en el modelo Jaccard ítem–ítem.

Umbral	Precision	Recall	F1	Vecinos prom.	% sin vecinos
0.0	0.5981	0.9990	0.7482	12.47	0.6607
0.2	0.6192	0.9578	0.7521	0.71	0.9120

El aumento del umbral permitió una mejora ligera en precision y F1-score, pero a costa de una reducción fuerte en la cobertura, ya que la tasa de casos sin vecinos pasó de 66% a más de 91%. Esto confirma que, también en el enfoque ítem–ítem, el parámetro más relevante en Jaccard es el umbral de similitud y no el número de vecinos.

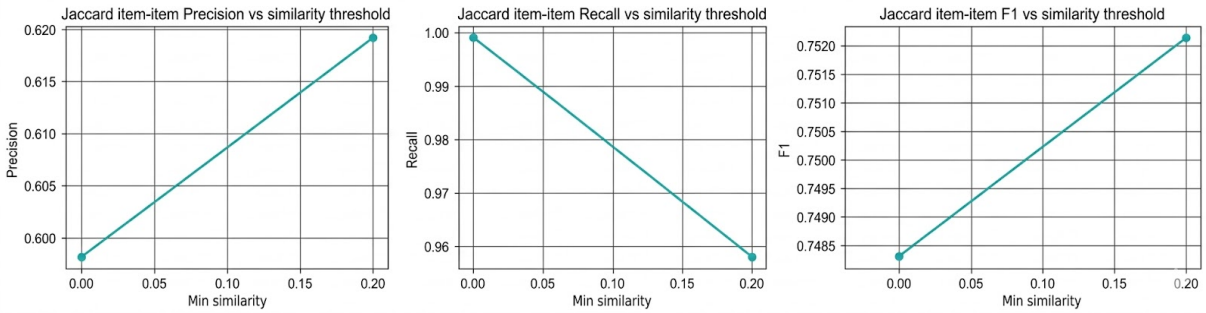

Figure 15: Impacto del umbral de similitud sobre precision, recall y F1-score en el modelo Jaccard ítem–ítem.

5.6. Ponderación por significancia de McLaughlin

Finalmente, se evaluó la estrategia de *significance weighting* en el contexto ítem–ítem, utilizando una configuración base fija por modelo y variando el parámetro β. A diferencia del enfoque usuario–usuario, el impacto de esta estrategia fue más heterogéneo y, en algunos casos, desfavorable.

5.6.1. Jaccard con significance weighting

En Jaccard ítem–ítem, la ponderación por significancia tuvo un efecto mínimo. Las métricas permanecieron prácticamente constantes para todos los valores de β, lo que sugiere que el modelo ya estaba dominado por la escasez de vecinos útiles y que la penalización adicional no alteró significativamente su comportamiento.

Tabla 21: Impacto de la significance weighting en Jaccard ítem–ítem.

β	Precision	Recall	F1	Vecinos prom.	% sin vecinos
10	0.6191	0.9574	0.7519	0.70	0.9140
20	0.6190	0.9572	0.7519	0.70	0.9144
40	0.6191	0.9572	0.7519	0.70	0.9150

5.6.2. Cosine y Pearson con significance weighting

En Cosine y Pearson ítem–ítem, la significance weighting produjo efectos distintos. En Cosine se observó una mejora en F1-score, pero a costa de un deterioro claro en RMSE y MAE. En Pearson, la penalización empeoró de forma importante tanto la calidad de predicción de rating como la clasificación de relevancia.

Tabla 22: Impacto de la significance weighting en los modelos Cosine y Pearson ítem–ítem.

Modelo	β	RMSE	MAE	F1	Vecinos prom.	% sin vecinos
Cosine	10	0.9977	0.7493	0.7674	38.15	0.0136
Cosine	20	1.0196	0.7663	0.7641	38.15	0.0136
Cosine	40	1.0378	0.7818	0.7614	38.15	0.0136
Pearson	10	1.0325	0.7905	0.6957	37.87	0.0145
Pearson	20	1.0316	0.7894	0.6969	37.87	0.0145
Pearson	40	1.0311	0.7883	0.6977	37.87	0.0145

En Cosine, la significance weighting generó un trade-off claro: mejoró la clasificación binaria, pero empeoró la precisión numérica del rating. En Pearson, el impacto fue claramente negativo, lo que sugiere que el modelo base ya era suficientemente robusto y que la penalización adicional eliminó parte de la información útil del vecindario.

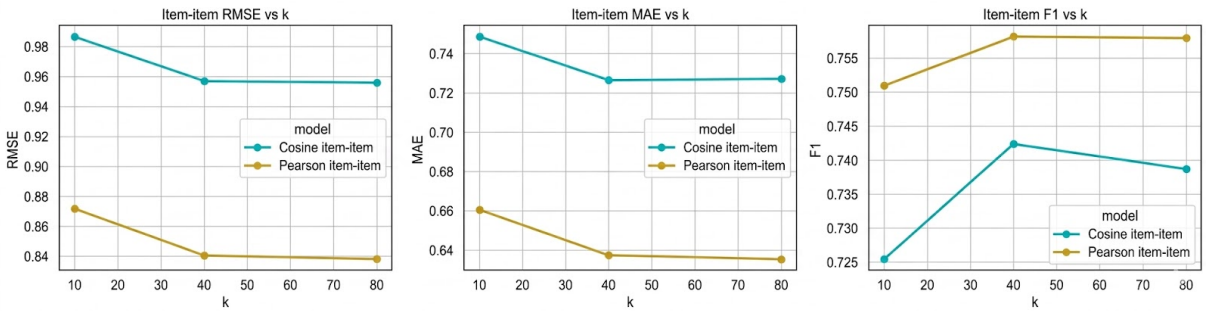

Figure 16: Impacto de β sobre F1-score, RMSE y MAE en los modelos ítem–ítem con significance weighting.

5.7. Conclusión del ajuste de parámetros en ítem–ítem

Los resultados muestran que el comportamiento del ajuste de parámetros en los modelos ítem–ítem es coherente con lo observado en usuario–usuario, pero con algunas diferencias importantes. En Jaccard, el parámetro más relevante siguió siendo el umbral de similitud, mientras que el número de vecinos apenas tuvo efecto. En Cosine y Pearson, el valor de k sí influyó en el rendimiento, aunque con mejoras que se estabilizan alrededor de $k = 40$.

En cuanto a la ponderación por significancia, su utilidad fue menor en el enfoque ítem–ítem que en el enfoque usuario–usuario. En Jaccard su efecto fue casi nulo; en Cosine produjo mejoras en F1, pero con deterioro en rating; y en Pearson resultó perjudicial en todas las métricas principales. En consecuencia, la mejor configuración dentro del esquema ítem–ítem se mantiene como **Pearson con $k = 40$ y sin significance weighting**.

6. Comparación final de los mejores modelos

Con el fin de sintetizar los resultados del taller, se seleccionó la mejor configuración encontrada para cada familia de modelos, considerando tanto el enfoque usuario–usuario como el enfoque ítem–ítem, así como los ajustes realizados por número de vecinos, umbral de similitud y significance weighting cuando correspondía.

La Figura 17 presenta la comparación final entre los mejores modelos, utilizando las métricas más relevantes del taller: F1-score para clasificación de relevancia y RMSE/MAE para predicción de rating.

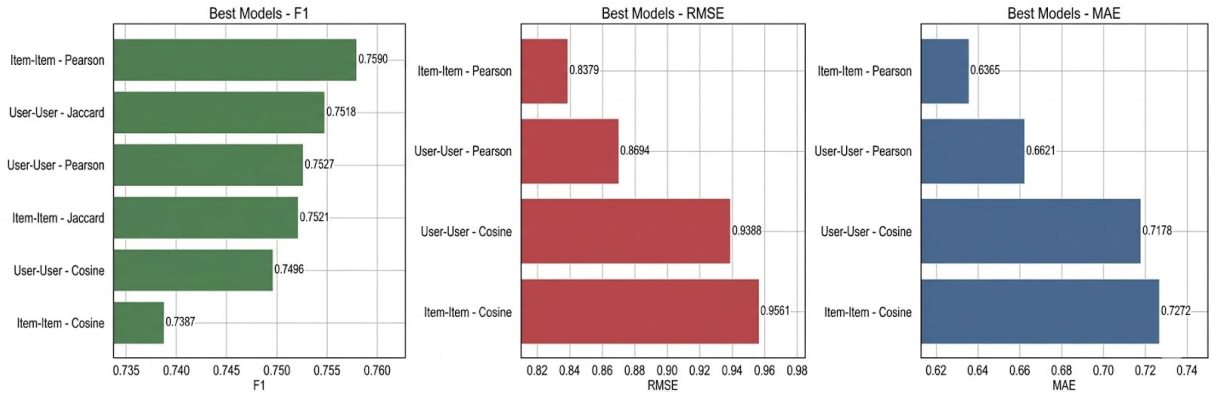

Figure 17: Comparación final entre los mejores modelos evaluados en el taller.

7. Conclusión final

El taller permitió comparar modelos de filtrado colaborativo usuario–usuario e ítem–ítem con las métricas Jaccard, Cosine y Pearson. En general, el desempeño dependió bastante de la medida de similitud utilizada.

Los modelos con **Jaccard** lograron un recall alto, pero tendieron a predecir demasiados positivos, lo que redujo su precisión. Aunque ajustar el umbral mejoró un poco los resultados, siguieron mostrando limitaciones.

Los modelos con **Cosine** tuvieron un comportamiento más estable y balanceado. Ofrecieron resultados aceptables tanto en usuario–usuario como en ítem–ítem, aunque sin ser los mejores en todos los casos.

El mejor resultado global se obtuvo con **Pearson ítem–ítem**, que logró el mejor desempeño tanto en predicción de rating como en F1-score entre los modelos evaluados. Esto sugiere que, en este taller, la similitud entre ítems fue más útil y estable que la similitud entre usuarios.

En conclusión, no existe una única mejor configuración para todos los objetivos. Jaccard puede ser útil si se busca alta recuperación, Cosine ofrece un buen equilibrio, y **Pearson ítem–ítem con** $k = 40$ **y sin significance weighting** fue la opción más sólida del taller.

Referencias

[1] GroupLens Research, *MovieLens 20M Dataset*, University of Minnesota. Disponible en la documentación oficial de MovieLens.

[2] N. Hug, *Surprise: A Python scikit for building and analyzing recommender systems*, documentación oficial de la librería Surprise.

[3] Matthew R. McLaughlin and Jonathan L. Herlocker, *A Collaborative Filtering Algorithm and Evaluation Metric that Accurately Model the User Experience*, Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 2004.